Team AERIS

AIORI 3

A3-PS004-TC336

Authenticated Gateway-Assisted DNSSEC For IoT Clients

Yash Sharma – yashtech0525@gmail.com
Parth Vaish – parthgupta5041@gmail.com
Tanish Sharma – rastogitanish673@gmail.com

AIORI Hackathon 3

October 5, 2026

Problem At Hand And Problem Understanding

- IoT devices like ESP32 microcontrollers have limited memory (520 KB RAM), processing power (240 MHz), and battery life, making them incapable of performing DNSSEC signature validation on their own.
- DNSSEC prevents DNS spoofing by validating cryptographic signature chains, but this verification requires significant computational resources that small IoT devices lack.



- Without DNSSEC validation, IoT devices are vulnerable to DNS spoofing attacks, Where attackers can redirect them to malicious servers.
- Existing solutions either require powerful hardware or leave IoT devices completely unprotected.
- There is a need for a delegated verification model where a resource-constrained IoT client can offload signature validation to a trusted local gateway.

Research Papers & Reference of Problem : https://pmc.ncbi.nlm.nih.gov/articles/PMC10181686/

Background

DNS vs DNSSEC

DNS is the foundational protocol that translates human-readable domain names into IP addresses, acting as the internet's phonebook. However, traditional DNS was designed without security in mind, making it highly vulnerable to spoofing and cache poisoning attacks. DNSSEC was introduced to address these weaknesses by adding cryptographic authentication and data integrity to DNS responses. The table below compares the two protocols across key dimensions.

Aspect	Traditional DNS	DNSSEC
Primary Function	Translates domain names into IP addresses	Adds security to DNS
Security	Lacks security — vulnerable to spoofing	Prevents spoofing and cache poisoning
Authentication	None	Data origin authentication
Integrity	None	Data integrity via cryptographic signatures
Key Records	A, AAAA, CNAME, MX	RRSIG, DNSKEY, DS, NSEC

DNSSEC Validation - Requirements

While DNSSEC provides strong security guarantees, it comes at a significant computational cost. Validating DNSSEC signatures requires substantial CPU cycles for cryptographic operations, sufficient memory to store keys and chain-of-trust data, and continuous battery power for ongoing validation. IoT devices such as smart bulbs, sensors, and smart locks are severely resource-constrained and cannot perform these operations without noticeable performance degradation.

Resource	Why DNSSEC Needs It	IoT Device Constraint
CPU Cycles	Translates domain names into IP addresses	Adds security to DNS
Memory	Lacks security — vulnerable to spoofing	Prevents spoofing and cache poisoning
Battery Power	None	Data origin authentication
Storage	None	Data integrity via cryptographic signatures
Network	A, AAAA, CNAME, MX	RRSIG, DNSKEY, DS, NSEC

Literature & RFC Review

Core DNSSEC Standards

- The foundational specifications for DNSSEC are defined in **RFC 4033, RFC 4034, and RFC 4035** . RFC 4033 introduces the security requirements and defines essential concepts such as the chain of trust and security-aware resolvers. RFC 4034 specifies the resource records used for cryptographic validation, including DNSKEY, RRSIG, DS, and NSEC. RFC 4035 describes the protocol modifications required for serving and resolving signed zones, including the DO and AD bits that signal validation status.
- RFC 6840 provides important clarifications for DNSSEC implementations, particularly regarding algorithm handling and trust anchor policies . It specifies that validating resolvers must disregard RRSIGs for algorithms not present in the zone's DNSKEY RRset and establishes the "Accept Any Success" policy for multiple trust anchors.
- RFC 9364 (BCP 237) serves as the consolidated reference for DNSSEC, stating that using DNSSEC for origin authentication is best current practice . It provides a single entry point to the DNSSEC RFC family, which is useful for documentation and reference purposes.

IoT DNS Security Guidelines

The IETF draft "IoT DNS Security and Privacy Guidelines" directly addresses the problem space of this project . It describes two validation models for IoT stub-resolvers: resolver validation (using the AD bit with explicit trust in the upstream resolver) and full local validation (cryptographic checks on the device). The draft notes that devices performing local validation gain end-to-end trust but at higher computational cost. This guideline validates the design choice of offloading validation to a trusted gateway when client resources are insufficient for full validation.

Academic Research

CSProp (USENIX Security 2021) proposes a cryptographic primitive that enables efficient public-key operations by propagating ciphertexts and signatures to end devices . The paper specifically applies CSProp to DNSSEC, demonstrating that propagated signature verification reduces latency by 78x and energy consumption by 47x on a Raspberry Pi Zero compared to traditional DNSSEC validation. This work provides quantitative evidence for the performance benefits of delegating DNSSEC validation to a gateway.

Patent Art (Prior Work) : US10057214B2 ("DNSSEC lightweight database access protocol gateway") describes a system that translates between LDAP and DNSSEC on-the-fly, spoofing a directory service to accelerate DNSSEC adoption . While the use case differs, the patent establishes prior art for the general concept of a gateway that performs DNSSEC operations on behalf of clients that cannot natively support the protocol.

Proposed Solution

Overview Solution

Our Solution is a delegated DNSSEC verification model where a resource-constrained IoT client offloads signature validation to a trusted gateway.

System Components

IoT Client

- Sends DNS Queries Over Authenticated DTLS.
- Does NOT perform DNSSEC validation itself.
- Receives only verified IP Addresses

↓

Validating Gateway (Unbound)

- Performs DNSSEC validation on behalf of client.
- Builds chain of trust from root to auth server.
- Rejects bogus DNS responses

↓

Backend & Dashboard

- Collects metrics (latency, CPU, memory).
- Stores data in MongoDB.
- Visualizes on React dashboard.

Security Mechanisms

Certificate Pinning Prevents MITM

Trust Anchor Root Key For DNSSEC

DTLS Encryption Secure Channel

Bogus Rejection Invalid Signatures

Workflow Summary

- **Step 1:** ESP32 Sends DNS Query over DTLS.
- **Step 2:** Gateway validates DNSSEC signatures.
- **Step 3:** Secure Response returned to client.
- **Step 4:** Metrics sent to backend.
- **Step 5:** Dashboard visualizes real-time data.

Optimization Proposed

Key Optimizations

Delegated Validation
Client no longer performs DNSSEC validation; gateway handles it.

Certificate Pinning
Prevents man-in-the-middle attacks; rejects fake gateways.

DTLS Encryption
Reduces repeated validation; lowers latency for frequent queries.

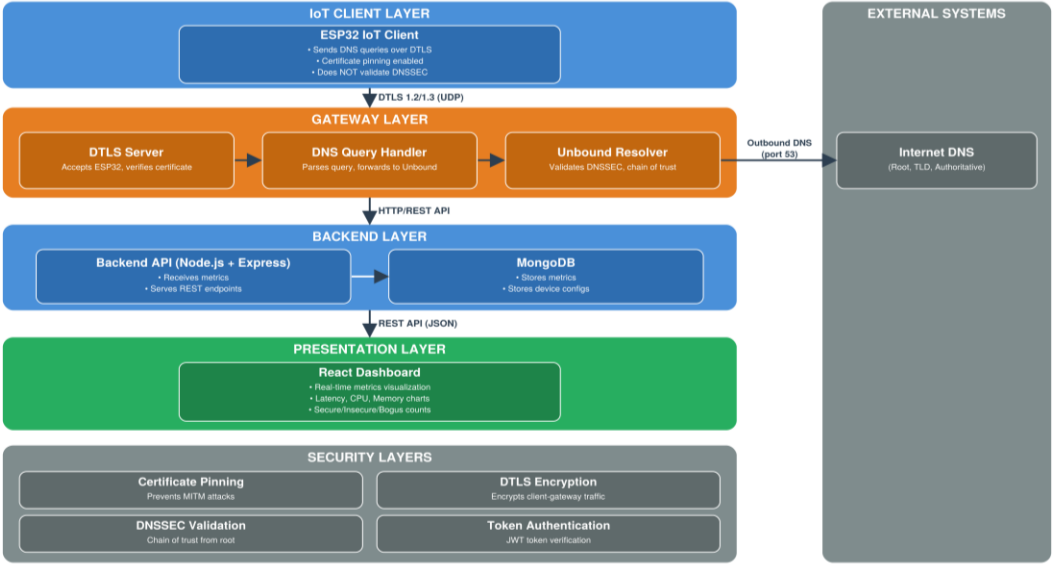
Gateway Caching
Reduces repeated validation; lowers latency for frequent queries.

Metrics Collection
Real-time monitoring of latency, CPU, and memory usage.

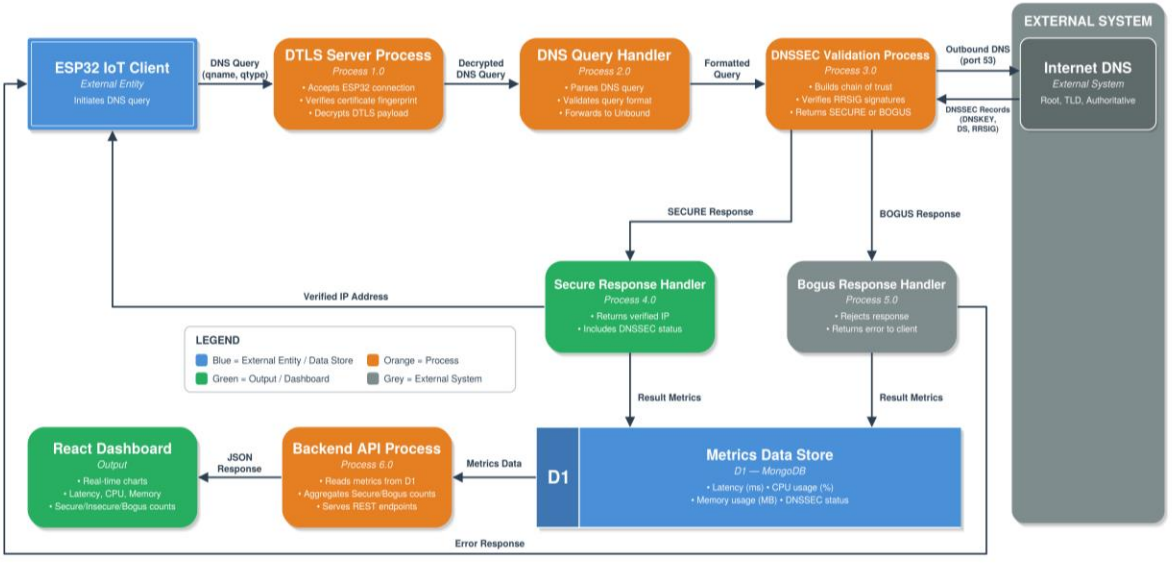
Scalability
Single gateway can handle multiple IoT clients simultaneously.

Aspect	Before (Without Gateway)	After (With Gateway)
DNSSEC Validation	Client performs validation	Gateway performs validation
CPU Usage	High (crypto operations)	Low (no crypto on client)
Memory Usage	High (keys + signatures)	Low (only query/response)
Battery Life	Short (continuous validation)	Long (delegated validation)
Latency	High (validation time)	Low (gateway caches results)
Security	Vulnerable to spoofing	Protected by DNSSEC

Solution Architecture & Design

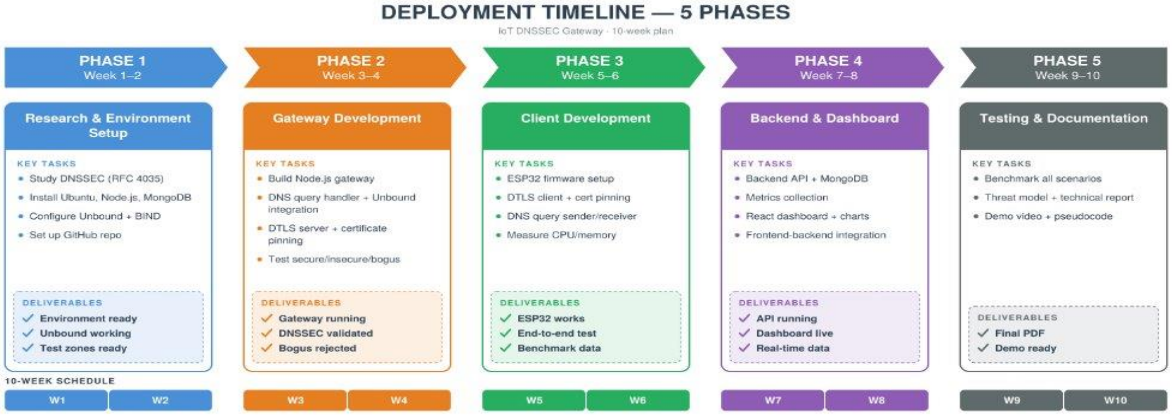


Data Flow Diagram



Deployment Timeline

The project is planned across five phases spanning ten weeks. Phase 1 focuses on research and environment setup, including studying DNSSEC (RFC 4035), installing Ubuntu, Node.js, MongoDB, and configuring Unbound and BIND. Phase 2 involves building the Node.js gateway, implementing DNS query handling, DTLS server, and certificate pinning. Phase 3 focuses on ESP32 client development, including DTLS client firmware, DNS query sender/receiver, and CPU/memory benchmarking. Phase 4 covers backend API development, MongoDB integration, and React dashboard visualization. Phase 5 includes comprehensive testing, threat model documentation, technical report preparation, and demo video recording.



Bibliography I

Ref #	Type	Citation	URL / Source
1	**RFC**	R. Arends et al., “Protocol Modifications for the DNS Security Extensions,” RFC 4035, Mar. 2005.	https://www.rfc-editor.org/info/rfc4035/
2	**RFC**	R. Arends et al., “DNS Security Introduction and Requirements,” RFC 4033, Mar. 2005.	https://www.rfc-editor.org/info/rfc4033/
3	**Software Tool**	NLnet Labs, “Unbound: Validating, Recursive, Caching DNS Resolver.”	https://nlnetlabs.nl/projects/unbound/about/
4	**Software Tool**	Internet Systems Consortium, “BIND 9 DNS Server.”	https://www.isc.org/bind/
5	**RFC**	R. Arends et al., “Resource Records for the DNS Security Extensions,” RFC 4034, Mar. 2005.	https://www.rfc-editor.org/info/rfc4034/
6	**Hardware Documentation**	Espressif Systems, “ESP32 Technical Reference Manual."	https://www.espressif.com/en/products/socs/esp32